Grade: 8.4

NUR Hand-in 2

Berend Nieuwhof

March 20, 2025

Abstract

In this document, the solutions to the second hand-in exercise are presented.

1 Exercise 1: Satellite galaxies

In this section, the solutions to the first exercise are presented.

Our script is given by:

```python
import numpy as np
import matplotlib.pyplot as plt
import copy

def n(x, A, Nsat, a, b, c):
    return A*Nsat*((x/b)**(a-3))*np.exp(-(x/b)**c)

def N(x, A, Nsat, a, b, c):
    return n(x, A, Nsat, a, b, c)*4*np.pi*x**2

def romberg_interval(func, a, b, order, *args):
    '''Calculates the integral of a function from a to b, using
    some order of Richardson extrapolation'''
    r = np.zeros(order)

    # calculate r0
    h = (b - a)
    r[0] = 0.5 * h * (func(a, *args) - func(b, *args))
    Np = 1
    for i in np.arange(1, order-1): # calculate approximations
        r[i] = 0
        delta = h
        h = 0.5 * h
        x = a + h
        for n in range(Np):
            r[i] += func(x, *args)
            x += delta
        r[i] = 0.5*(r[i-1] + delta*r[i])
        Np = 2*Np
    Np = 1
    for i in np.arange(1, order-1): # combine approximations. r[0] holds the best one
        Np = 4*Np
        for j in np.arange(0, order-i):
            r[j] = (Np * r[j+1] - r[j]) / (Np - 1)

    return r

def romberg(func, x, order, *args):
    '''with x a linspace. calculates the integral over a range given by x.
    len(x) is the number of function evaluations we use to integrate.'''
    result = 0
    for i in range(len(x)-1):
        a = x[i]
        b  = x[i+1]
        result += romberg_interval(func, a, b, order, *args)[0]
    return result
```

```
47
48 A=1. # to be computed
49 Nsat=100
50 a=2.4
51 b=0.25
52 c=1.6
53
54 # We want to integrate the function (with A = 1) to find what we need to set A to
55 # to find <Nsat> = 100. A will be 100 divided by the value of the integral.
56
57 # Because n(x) is radially symmetric, we can use the fact that
58 # dV = 4pi*r**2 dr to write down the integral.
59
60 x = np.linspace(0.00001, 5, 15)
61 integ_result = romberg(N, x, 10, A, 100, a, b, c)
62 new_A = Nsat / integ_result
63 print('A found by integral normalisation:', new_A)
64
65 check = romberg(N, x, 10, new_A, Nsat, a, b, c)
66 check = Nsat / check
67 print('Sanity check:', check)
68
69 def xor_shift(seed, a1 = 21, a2 = 35, a3 = 4):
70     '''Performs 64bit xor shift on the seed number.'''
71     state = np.uint64(seed)
72     a1 = np.uint64(a1)
73     a2 = np.uint64(a2)
74     a3 = np.uint64(a3)
75
76     state = state ^ (state >> a1)
77     state = state ^ (state << a2)
78     state = state ^ (state >> a3)
79     return state
80
81 def mult_w_carry(seed, a = 4294957665):
82     '''Performs multiply with carry, base 2**32'''
83     state = np.uint64(seed)
84     a = np.uint64(a)
85
86     state = a*(state & np.uint64(2**32 - 1)) + (state >> np.uint64(32))
87     number = state & np.uint64(2**32 - 1)
88
89     return number, state
90
91 def rng_int(seed, size):
92     state = np.uint64(seed)
93     result = np.zeros(size)
94     for i in range(size):
95         xor_state = xor_shift(state)
96         if xor_state < 2**32:
97             number, mult_state = mult_w_carry(xor_state)
98         else:
99             number, mult_state = mult_w_carry(xor_state & np.uint64(2**32 - 1))
100        state = mult_state
101        result[i] = number
102
103    return result
104
105 def rng_float(seed, size, a, b):
106     ints = rng_int(seed, size)
107     floats = a + (b-a)*(ints / (2**32))
108     return floats
109
110 def mergesort(a):
111     N = len(a)
112     idx = np.arange(0, N, 1)
113     def mergesort_helper(idx):
114         if len(idx) > 1:
115             mid_idx = len(idx)//2
116             left = copy.deepcopy(idx[:mid_idx])
```

```
            right = copy.deepcopy(idx[mid_idx:])

            # recursively sort the left and right arrays
            mergesort_helper(left)
            mergesort_helper(right)
            i, j, k = 0, 0, 0
            # go through the arrays and check which element of is the smallest between them, then add that one to a
            while i < len(left) and j < len(right):
                if a[left[i]] < a[right[j]]:
                    idx[k] = left[i]
                    i += 1
                else:
                    idx[k] = right[j]
                    j += 1
                k += 1
            # if left or right still hold elements after one of the lists is ran through, put them into a in order
            while i < len(left):
                idx[k] = left[i]
                i += 1
                k += 1
            while j < len(right):
                idx[k] = right[j]
                j += 1
                k += 1

    mergesort_helper(idx)
    return idx


#Plot of histogram in log-log space with line (question 1b)
xmin, xmax = 10**-4, 5
N_generate = 10000

# Make the analytical function
relative_radius = np.linspace(xmin, xmax, 10000)
analytical_function = N(relative_radius, new_A, Nsat, a, b, c) / Nsat

# find the maximum value by sorting
sorting_idx2 = mergesort(analytical_function)
sorted_analytical = analytical_function[sorting_idx2]
max_val2 = sorted_analytical[-1]

# Do rejection sampling for r.
Pu3 = 10**rng_float(44, N_generate*10, np.log10(xmin), np.log10(xmax))
Pu4 = rng_float(33, N_generate*10, 0, max_val2)
P_x = N(Pu3, new_A, Nsat, a, b, c) / Nsat

rej_result = []
for i in range(N_generate*10):
    if Pu4[i] < P_x[i]:
        rej_result.append(Pu3[i])
        if len(rej_result) == 10000:
            break
rej_result = np.array(rej_result) # take the first 10000 elements

#21 edges of 20 bins in log-space
edges = 10**np.linspace(np.log10(xmin), np.log10(xmax), 21)
hist = np.histogram(rej_result, bins=edges)[0]
hist_scaled = Nsat * hist / (len(Pu3)) # normalize by the amount of samples originally used.

hist_px = np.histogram(P_x, bins = edges)[0]
hist_px_scaled = Nsat * hist / (len(P_x))

fig1b, ax = plt.subplots()
# ax.stairs(hist_px_scaled, edges = edges, fill=False, label = 'calculated Px')
ax.stairs(hist_scaled, edges=edges, fill=False, label='Satellite galaxies')
plt.plot(relative_radius, analytical_function, 'r-', label='Analytical solution')
```

```
ax.set(xlim=(xmin, xmax), ylim=(10**(-3), 10), yscale='log', xscale='log',
       xlabel='Relative radius', ylabel='Number of galaxies')
ax.legend()
plt.savefig('my_solution_1b.png', dpi=600)
plt.close()

# We'll achieve this by generating a random sequence using our random number generator.
# We will then sort the array and thus generate a shuffle index which we then apply to
# the samples from above.
rnd_seq = rng_float(69, 10000, 0, 1)
rnd_shuffle_idx = mergesort(rnd_seq)

# Select 100 random satellite galaxies by applying the index to the samples
# and then selecting the first 100
chosen = rej_result[rnd_shuffle_idx][:100]
sorting_idx = mergesort(chosen)
chosen_sorted = chosen[sorting_idx]

fig1c, ax = plt.subplots()
ax.plot(chosen_sorted, np.arange(100))
ax.set(xscale='log', xlabel='Relative radius',
       ylabel='Cumulative number of galaxies',
       xlim=(xmin, xmax), ylim=(0, 100))
plt.savefig('my_solution_1c.png', dpi=600)

# Now we will calculate dn(x)/dx at x = 1.

def n_deriv(x, A, Nsat, a, b, c):
    return A*Nsat/b * ((a-3)*(x/b)**(a-4) - c*(x/b)**(a+c-4))*np.exp(-(x/b)**c)

def central_diff(x, func, h, *args):
    deriv = (func(x + h, *args) - func(x - h, *args)) / (2*h)
    return deriv

def ridder(x, func, h, d, *args, order = 5, tol = 1e-8):
    r = np.zeros(order)
    oneDeeth = 1 / d
    r[0] = central_diff(x, func, h, *args)
    for o in range(1, order):
        h = h * oneDeeth
        r[o] = central_diff(x, func, h, *args)
    # now iterate to combine the previous

    for j in range(1, order):
        for i in range(order-1-j):
            r[i] = (d**(2*j) * r[i+1] - r[i]) / (d**(2*j) - 1)
    return r[0]

#Cumulative plot of the chosen galaxies (1c)
x = 1
h = 0.2

deriv_x = ridder(x, n, h, 2, new_A, Nsat, a, b, c, order = 10)
deriv_x_a = n_deriv(x, new_A, Nsat, a, b, c)
print('derivative of n with respect to x')
print("with Ridder's method:", deriv_x)
print('Analytically:', deriv_x_a)
```

Satellites.py

1.1 1a. Finding A by numerical integration.

In order to find the normalization constant A in the formula given in the problem sheet, we implemented a Romberg integration algorithm. For the integral, we use $dV = 4\pi r^2 dr$. For the rest of this exercise, $n(x)$ will refer to the number density, and $N(x)$ will be the number density multiplied by $4\pi r^2$. The relevant code is given below:

```

```

```python
def n(x, A, Nsat, a, b, c):
    return A*Nsat*((x/b)**(a-3))*np.exp(-(x/b)**c)

def N(x, A, Nsat, a, b, c):
    return n(x, A, Nsat, a, b, c)*4*np.pi*x**2

def romberg_interval(func, a, b, order, *args):
    '''Calculates the integral of a function from a to b, using
    some order of Richardson extrapolation'''
    r = np.zeros(order)

    # calculate r0
    h = (b - a)
    r[0] = 0.5 * h * (func(a, *args) + func(b, *args))
    Np = 1
    for i in np.arange(1, order-1): # calculate approximations
        r[i] = 0
        delta = h
        h = 0.5 * h
        x = a + h
        for n in range(Np):
            r[i] += func(x, *args)
            x += delta
        r[i] = 0.5*(r[i-1] + delta*r[i])
        Np = 2*Np
    Np = 1
    for i in np.arange(1, order-1): # combine approximations. r[0] holds the best one
        Np = 4*Np
        for j in np.arange(0, order-i):
            r[j] = (Np * r[j+1] - r[j]) / (Np - 1)

    return r

def romberg(func, x, order, *args):
    '''with x a linspace. calculates the integral over a range given by x.
    len(x) is the number of function evaluations we use to integrate.'''
    result = 0
    for i in range(len(x)-1):
        a = x[i]
        b  = x[i+1]
        result += romberg_interval(func, a, b, order, *args)[0]
    return result

A=1. # to be computed
Nsat=100
a=2.4
b=0.25
c=1.6

# We want to integrate the function (with A = 1) to find what we need to set A to
# to find <Nsat> = 100. A will be 100 divided by the value of the integral.

# Because n(x) is radially symmetric, we can use the fact that
# dV = 4pi*r**2 dr to write down the integral.

x = np.linspace(0.00001, 5, 15)
integ_result = romberg(N, x, 10, A, 100, a, b, c)
new_A = Nsat / integ_result
print('A found by integral normalisation:', new_A)

check = romberg(N, x, 10, new_A, Nsat, a, b, c)
check = Nsat / check
print('Sanity check:', check)
```

Satellites.py

As can be seen in the code above, romberg_interval calculates the integral between a and b up to some order, and romberg then sums this over many intervals. In this case, we go up to order 10, where we use 15 function evaluations. The output given by this piece of the code is given below. For the sanity check, we calculate the integral again with our calculated value of A, and we see that it is equal to one.

```
A found by integral normalisation: 9.211900153463198
Sanity check: 1.0000000000000007
```

1a: 2.5/4

satellite_output.txt

1.2 1b. Generating satellite positions

For this subquestion, we made use of rejection sampling in log space to sample 10,000 points following the analytical distribution $N(x)dx/\langle N_{sat}\rangle$. The relevant code is given below.

```python
def xor_shift(seed, a1 = 21, a2 = 35, a3 = 4):
    '''Performs 64bit xor shift on the seed number.'''
    state = np.uint64(seed)
    a1 = np.uint64(a1)
    a2 = np.uint64(a2)
    a3 = np.uint64(a3)

    state = state ^ (state >> a1)
    state = state ^ (state << a2)
    state = state ^ (state >> a3)
    return state

def mult_w_carry(seed, a = 4294957665):
    '''Performs multiply with carry, base 2**32'''
    state = np.uint64(seed)
    a = np.uint64(a)

    state = a*(state & np.uint64(2**32 - 1)) + (state >> np.uint64(32))
    number = state & np.uint64(2**32 - 1)

    return number, state

def rng_int(seed, size):
    state = np.uint64(seed)
    result = np.zeros(size)
    for i in range(size):
        xor_state = xor_shift(state)
        if xor_state < 2**32:
            number, mult_state = mult_w_carry(xor_state)
        else:
            number, mult_state = mult_w_carry(xor_state & np.uint64(2**32 - 1))
        state = mult_state
        result[i] = number

    return result

def rng_float(seed, size, a, b):
    ints = rng_int(seed, size)
    floats = a + (b-a)*(ints / (2**32))
    return floats

def mergesort(a):
    N = len(a)
    idx = np.arange(0, N, 1)
    def mergesort_helper(idx):
        if len(idx) > 1:
            mid_idx = len(idx)//2
            left = copy.deepcopy(idx[:mid_idx])
            right = copy.deepcopy(idx[mid_idx:])

            # recursively sort the left and right arrays
            mergesort_helper(left)
            mergesort_helper(right)
            i, j, k = 0, 0, 0
            # go through the arrays and check which element of is the smallest between them, then add that one to a
            while i < len(left) and j < len(right):
                if a[left[i]] < a[right[j]]:
```

```
                    idx[k] = left[i]
                    i += 1
                else:
                    idx[k] = right[j]
                    j += 1
                k += 1
            # if left or right still hold elements after one of the lists is ran through, put them into a in order
            while i < len(left):
                idx[k] = left[i]
                i += 1
                k += 1
            while j < len(right):
                idx[k] = right[j]
                j += 1
                k += 1

    mergesort_helper(idx)
    return idx


#Plot of histogram in log-log space with line (question 1b)
xmin, xmax = 10**-4, 5
N_generate = 10000

# Make the analytical function
relative_radius = np.linspace(xmin, xmax, 10000)
analytical_function = N(relative_radius, new_A, Nsat, a, b, c) / Nsat

# find the maximum value by sorting
sorting_idx2 = mergesort(analytical_function)
sorted_analytical = analytical_function[sorting_idx2]
max_val2 = sorted_analytical[-1]

# Do rejection sampling for r.
Pu3 = 10**rng_float(44, N_generate*10, np.log10(xmin), np.log10(xmax))
Pu4 = rng_float(33, N_generate*10, 0, max_val2)
P_x = N(Pu3, new_A, Nsat, a, b, c) / Nsat

rej_result = []
for i in range(N_generate*10):
    if Pu4[i] < P_x[i]:
        rej_result.append(Pu3[i])
        if len(rej_result) == 10000:
            break
rej_result = np.array(rej_result) # take the first 10000 elements

#21 edges of 20 bins in log-space
edges = 10**np.linspace(np.log10(xmin), np.log10(xmax), 21)
hist = np.histogram(rej_result, bins=edges)[0]
hist_scaled = Nsat * hist / (len(Pu3)) # normalize by the amount of samples originally used.

hist_px = np.histogram(P_x, bins = edges)[0]
hist_px_scaled = Nsat * hist / (len(P_x))

fig1b, ax = plt.subplots()
# ax.stairs(hist_px_scaled, edges = edges, fill=False, label = 'calculated Px')
ax.stairs(hist_scaled, edges=edges, fill=False, label='Satellite galaxies')
plt.plot(relative_radius, analytical_function, 'r-', label='Analytical solution')
ax.set(xlim=(xmin, xmax), ylim=(10**(-3), 10), yscale='log', xscale='log',
       xlabel='Relative radius', ylabel='Number of galaxies')
ax.legend()
plt.savefig('my_solution_1b.png', dpi=600)
plt.close()
```

Satellites.py

The analytical function is equal to $N(x) = n(x)4\pi r^2$. We first find the maximum this function takes on in our interval, by evaluating the function at many points (which is already necessary for plotting),

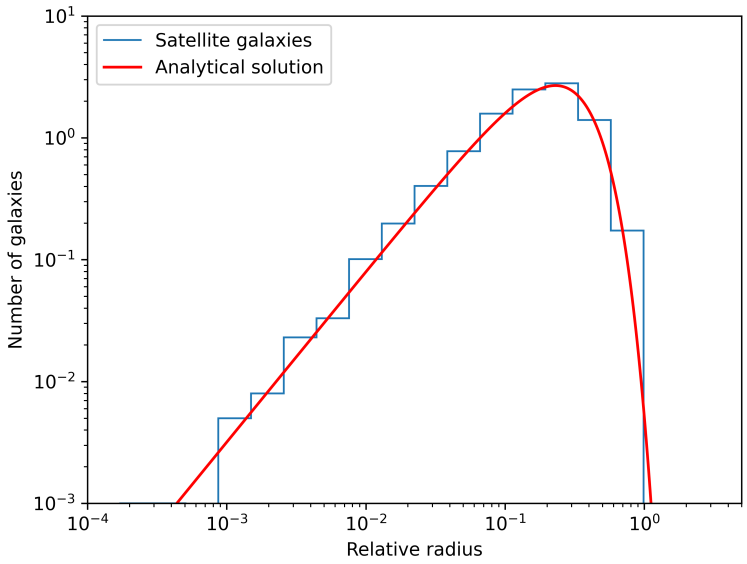
sorting the array of evaluations, and finding the first element of it. We use this maximum as the maximum for our uniform random numbers.

Speaking of uniform random numbers, we generate ours as follows: we feed a number into a xor_shift algorithm, and the result of that is put into a Multiply With Carry random number generator. The MWC returns both a number and a state: the number is output as a random number, and the state is then fed back into the xor_shift algorithm, where the next random number will be generated. This of course generates integers between 0 and 2^{32}, so we use the rng_float function to generate uniform random floats between two values.

We use rejection sampling to sample the distribution, and we do so by sampling our x value in log space, to match the logarithmic bins given by the exercise instructions. We then compare another random number $y = U(0, max)$, with max being the maximum calculated earlier, to the function evaluated at our x value. If y is smaller than x, we accept y as part of our sample. We generate samples until we've reached 10,000, at which point we stop generating them.

The result of this sampling is plotted below. Note that we normalize our histogram by dividing by the total amount of samples generated to reach 10,000 accepted samples, and multiplying by the average number of satellites.

Figure 1: In red, the analytical distribution that we are supposed to sample from is plotted. The blue bars correspond to the logarithmic bins given, and their heights are the number of samples per bin. The bars follow the distribution nicely.



1b: 3.5/6

1.3 1c. Random galaxy selection

In this subquestion, we randomly select satellite galaxies from the sample in b in a way that selects every galaxy with equal probability, does not draw the same galaxy twice, and does not reject any draw. The relevant code is included here. All sorting in this exercise is done with mergesort, which can be found in the first code snippet in this exercise.

```
1
2 # We'll achieve this by generating a random sequence using our random number generator.
3 # We will then sort the array and thus generate a shuffle index which we then apply to
4 # the samples from above.
5 rnd_seq = rng_float(69, 10000, 0, 1)
6 rnd_shuffle_idx = mergesort(rnd_seq)
7
8 # Select 100 random satellite galaxies by applying the index to the samples
9 # and then selecting the first 100
```

```
chosen = rej_result[rnd_shuffle_idx][:100]
sorting_idx = mergesort(chosen)
chosen_sorted = chosen[sorting_idx]

fig1c, ax = plt.subplots()
ax.plot(chosen_sorted, np.arange(100))
ax.set(xscale='log', xlabel='Relative radius',
       ylabel='Cumulative number of galaxies',
       xlim=(xmin, xmax), ylim=(0, 100))
plt.savefig('my_solution_1c.png', dpi=600)
```

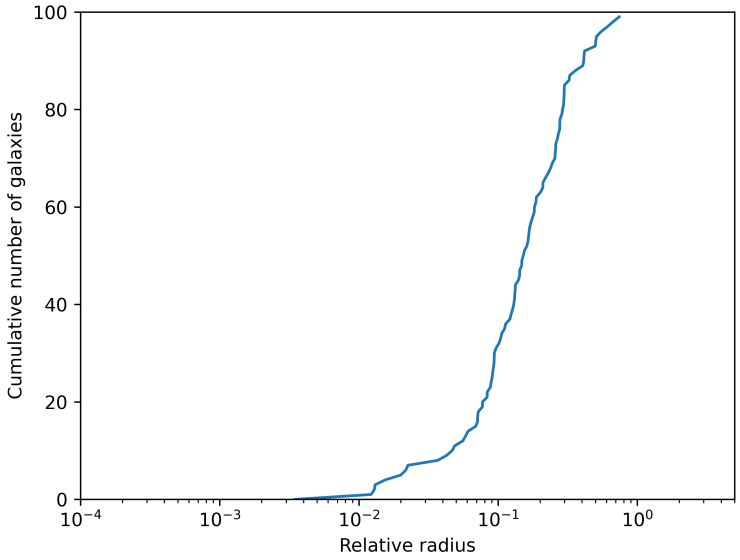
Satellites.py

We achieve this by generating an array of 10,000 random numbers, and sorting them to create a random sorting index. We then apply this sorting index to the random sample from b, and select the first 100 elements. This achieves the requirements specified above.

Then, we sort the 100 galaxies from smallest to largest radius using our implementation of mergesort, and plot the number of galaxies within a radius.

Figure 2: We see that the number of galaxies within a radius is strictly increasing, which means the array is sorted correctly.

3/3



1.4 1d. Derivative of n(x)

In this section, we calculate the derivative $dn(x)/dx$ at $x = 1$ numerically. We used Ridder's algorithm to find the derivative, as seen in the code below.

```
# Now we will calculate dn(x)/dx at x = 1.

def n_deriv(x, A, Nsat, a, b, c):
    return A*Nsat/b * ((a-3)*(x/b)**(a-4) - c*(x/b)**(a+c-4))*np.exp(-(x/b)**c)

def central_diff(x, func, h, *args):
    deriv = (func(x + h, *args) - func(x - h, *args)) / (2*h)
    return deriv

def ridder(x, func, h, d, *args, order = 5, tol = 1e-8):
    r = np.zeros(order)
    oneDeeth = 1 / d
    r[0] = central_diff(x, func, h, *args)
```

```
    for o in range(1, order):
        h = h * oneDeeth
        r[o] = central_diff(x, func, h, *args)
    # now iterate to combine the previous

    for j in range(1, order):
        for i in range(order-1-j):
            r[i] = (d**(2*j) * r[i+1] - r[i]) / (d**(2*j) - 1)
    return r[0]

#Cumulative plot of the chosen galaxies (1c)
x = 1
h = 0.2

deriv_x = ridder(x, n, h, 2, new_A, Nsat, a, b, c, order = 10)
deriv_x_a = n_deriv(x, new_A, Nsat, a, b, c)
print('derivative of n with respect to x')
print("with Ridder's method:", deriv_x)
print('Analytically:', deriv_x_a)
```

Satellites.py

The output is given here:

```
derivative of n with respect to x
with Ridder's method: -0.6264877290898574
Analytically: -0.6264877290898926
```

6/6

satellite_output.txt

2 Exercise 2: HII-regions

In this section we look at exercise 2, concerning equilibrium temperatures in HII-regions under various healing and cooling influences. We make use of the functions provided, equilibrium1 and equilibrium2. The code for the full exercise is given below:

```
import numpy as np
import matplotlib.pyplot as plt
import copy
import timeit

def bisection(func, brac, *args, abs_tol = 1e-8, frac_tol = 1e-8, max_iter = 50):
    '''implements the bisection method of root finding.'''
    i = 0
    a = brac[0]
    b = brac[1]
    # check if a, b is a bracket
    # assert func(a, *args)*func(b, *args) < 0

    while i < max_iter:
        # Divide the bracket in 2
        c = (a+b)/2
        if func(a, *args)*func(c, *args) < 0:
            b = c
        else:
            a = c
        i += 1
        if abs(a-b) < abs_tol or abs((a-b)/a) < frac_tol:
            return a, i
    return a, i

def secant(func, guesses, *args, abs_tol = 1e-10, frac_tol = 1e-10, max_iter = 50):
    '''implements the secant method for root finding.'''
    i = 2
    # r will hold all guesses
    r = np.zeros(max_iter)
    r[0] = guesses[0]
    r[1] = guesses[1]
```

```
    while i < max_iter:
        r[i] = r[i-1] - ((r[i-1] - r[i-2])/(func(r[i-1], *args) - func(r[i-2], *args)))
    * func(r[i-1], *args)
        if abs(r[i] - r[i-1]) < abs_tol or abs((r[i] - r[i-1])/r[i]) < frac_tol:
            return r[i], i
        i += 1

    print('maximum secant interations reached')
    return r[i-1], i

def false_position(func, brac, *args, abs_tol = 1e-10, frac_tol = 1e-10, max_iter = 50):
    '''implements the false position method for root finding.'''
    i = 2
    r = np.zeros(max_iter)
    r[0] = brac[0]
    r[1] = brac[1]
    # check if a, b is a bracket
    assert func(r[0], *args)*func(r[1], *args) < 0

    while i < max_iter:
        r[i] = r[i-1] - ((r[i-1] - r[i-2])/(func(r[i-1], *args) - func(r[i-2], *args)))
    * func(r[i-1], *args)
        if func(r[i], *args)*func(r[i-2], *args) < 0:
            r[i-1] = r[i-2]
        if abs(r[i] - r[i-1]) < abs_tol or abs((r[i] - r[i-1])/r[i]) < frac_tol:
            return r[i], i
        i += 1

    return r[i-1], i

## 2a.

k=1.38e-16 # erg/K
aB = 2e-13 # cm^3 / s
Z = 0.015 # unitless
psi = 0.929 # unitless
Tc = 1e4 # K

# here no need for nH nor ne as they cancel out
def equilibrium1(T,Z,Tc,psi):
    return psi*Tc*k - (0.684 - 0.0416 * np.log(T/(1e4 * Z*Z)))*T*k

# test and visualize first
temps = np.linspace(1, 1e7, 1000)
eqs = equilibrium1(temps, Z, Tc, psi)
# plt.plot(temps, eqs)
# plt.show()

brac1 = [1, 1e7] # K
midpoint1 = 0.5e7 # mid point of the bracket interval
print('----------2a----------')
def time_bisection():
    root1, num_it1 = bisection(equilibrium1, brac1, Z, Tc, psi, abs_tol = 0.01, frac_tol
    = 1e-8, max_iter = 50)
    return root1, num_it1

def time_secant():
    root2, num_it2 = secant(equilibrium1, brac1, Z, Tc, psi, abs_tol = 0.01, frac_tol =
    1e-8, max_iter = 50)
    return root2, num_it2

def time_false_position():
    root3, num_it3 = false_position(equilibrium1, brac1, Z, Tc, psi, abs_tol = 0.01,
    frac_tol = 1e-8, max_iter = 50)
    return root3, num_it3

root1, num_it1 = bisection(equilibrium1, brac1, Z, Tc, psi, abs_tol = 0.01, frac_tol = 1
    e-8, max_iter = 50)
bisection_time = timeit.timeit(lambda : time_bisection(), number = 1)
print('\nroot and number of iterations')
```

```
print('using bisection:', root1, num_it1)
print('function evaluated at root:', equilibrium1(root1, Z, Tc, psi))
print('time taken:', bisection_time)

root2, num_it2 = secant(equilibrium1, brac1, Z, Tc, psi, abs_tol = 0.01, frac_tol = 1e-8, max_iter = 50)
secant_time = timeit.timeit(lambda : time_secant(), number = 1)
print('\nusing secant method:', root2, num_it2)
print('function evaluated at root:', equilibrium1(root2, Z, Tc, psi))
print('time taken:', secant_time)

root3, num_it3 = false_position(equilibrium1, brac1, Z, Tc, psi, abs_tol = 0.01, frac_tol = 1e-8, max_iter = 50)
false_position_time = timeit.timeit(lambda : time_false_position(), number = 1)
print('\nusing false position method:,', root3, num_it3)
print('function evaluated at root:', equilibrium1(root3, Z, Tc, psi))
print('time_taken', false_position_time)

def equilibrium2(T,Z,Tc,psi, nH, A, xi):
    return (psi*Tc - (0.684 - 0.0416 * np.log(T/(1e4 * Z*Z)))*T - .54 * ( T/1e4 )**.37 * T)*k*nH*aB + A*xi + 8.9e-26 * (T/1e4)


print('----------------2b----------------')
# 2b
A = 5e-10 # erg
xi = 1e-15 # s**-1
brac2 = [1, 1e15] # K
k=1.38e-16 # erg/K
aB = 2e-13 # cm^3 / s
Z = 0.015 # unitless
psi = 0.929 # unitless
Tc = 1e4 # K

# test and visualize first
temps = np.linspace(1, 1e15, 1000)
ne = 1e5
eqs = equilibrium2(temps, Z, Tc, psi, ne, A, xi)
# plt.plot(temps, eqs)
# plt.show()

for ne in [1e-4, 1, 1e4]:
    print('\nfor ne = ', ne)
    root1, num_it1 = bisection(equilibrium2, brac2, Z, Tc, psi, ne, A, xi, abs_tol = 1e-10, frac_tol = 1e-10, max_iter = 100)
    print('root and number of iterations')
    print('using bisection:', root1, num_it1)
    print('function evaluated at root:', equilibrium2(root1, Z, Tc, psi, ne, A, xi))

    root2, num_it2 = secant(equilibrium2, brac2, Z, Tc, psi, ne, A, xi, abs_tol = 1e-10, frac_tol = 1e-10, max_iter = 100)
    print('using secant method:', root2, num_it2)
    print('function evaluated at root:', equilibrium2(root2, Z, Tc, psi, ne, A, xi))

    root3, num_it3 = false_position(equilibrium2, brac2, Z, Tc, psi, ne, A, xi, abs_tol = 1e-10, frac_tol = 1e-10, max_iter = 100000)
    print('using false position method:,', root3, num_it3)
    print('function evaluated at root:', equilibrium2(root3, Z, Tc, psi, ne, A, xi))

print('----------------------------------')
def smallest_ne():
    ne = 1e-4
    root1, num_it1 = bisection(equilibrium2, brac2, Z, Tc, psi, ne, A, xi, abs_tol = 1e-10, frac_tol = 1e-10, max_iter = 100)
    return root1, num_it1

def middle_ne():
    ne = 1
    root2, num_it2 = secant(equilibrium2, brac2, Z, Tc, psi, ne, A, xi, abs_tol = 1e-10, frac_tol = 1e-10, max_iter = 100)
```

```
    return root2, num_it2
def big_ne():
    ne = 1e4
    root2, num_it2 = secant(equilibrium2, brac2, Z, Tc, psi, ne, A, xi, abs_tol = 1e-10,
     frac_tol = 1e-10, max_iter = 100)
    return root2, num_it2

small_ne_time = timeit.timeit(lambda : smallest_ne(), number = 1)
mid_ne_time = timeit.timeit(lambda : middle_ne(), number = 1)
big_ne_time = timeit.timeit(lambda : big_ne(), number = 1)

print('Bisection for smallest n took:', small_ne_time)
print('Secant for middle ne took:', mid_ne_time)
print('Secant for biggest ne took:', big_ne_time)
```

HII–regions.py

2.1 2a: A simple HII-region

For this exercise, we have to find the root of the equilibrium1 function that was provided. We time and test 3 root finding algorithms, to find the fastest one. The relevant code is given below:

```
import numpy as np
import matplotlib.pyplot as plt
import copy
import timeit

def bisection(func, brac, *args, abs_tol = 1e-8, frac_tol = 1e-8, max_iter = 50):
    '''implements the bisection method of root finding.'''
    i = 0
    a = brac[0]
    b = brac[1]
    # check if a, b is a bracket
    # assert func(a, *args)*func(b, *args) < 0

    while i < max_iter:
        # Divide the bracket in 2
        c = (a+b)/2
        if func(a, *args)*func(c, *args) < 0:
            b = c
        else:
            a = c
        i += 1
        if abs(a-b) < abs_tol or abs((a-b)/a) < frac_tol:
            return a, i
    return a, i

def secant(func, guesses, *args, abs_tol = 1e-10, frac_tol = 1e-10, max_iter = 50):
    '''implements the secant method for root finding.'''
    i = 2
    # r will hold all guesses
    r = np.zeros(max_iter)
    r[0] = guesses[0]
    r[1] = guesses[1]
    while i < max_iter:
        r[i] = r[i-1] - ((r[i-1] - r[i-2])/(func(r[i-1], *args) - func(r[i-2], *args)))
    * func(r[i-1], *args)
        if abs(r[i] - r[i-1]) < abs_tol or abs((r[i] - r[i-1])/r[i]) < frac_tol:
            return r[i], i
        i += 1

    print('maximum secant interations reached')
    return r[i-1], i

def false_position(func, brac, *args, abs_tol = 1e-10, frac_tol = 1e-10, max_iter = 50):
    '''implements the false position method for root finding.'''
    i = 2
    r = np.zeros(max_iter)
    r[0] = brac[0]
```

```
    r[1] = brac[1]
    # check if a, b is a bracket
    assert func(r[0], *args)*func(r[1], *args) < 0

    while i < max_iter:
        r[i] = r[i-1] - ((r[i-1] - r[i-2])/(func(r[i-1], *args) - func(r[i-2], *args)))
    * func(r[i-1], *args)
        if func(r[i], *args)*func(r[i-2], *args) < 0:
            r[i-1] = r[i-2]
        if abs(r[i] - r[i-1]) < abs_tol or abs((r[i] - r[i-1])/r[i]) < frac_tol:
            return r[i], i
        i += 1

    return r[i-1], i

## 2a.

k=1.38e-16 # erg/K
aB = 2e-13 # cm^3 / s
Z = 0.015 # unitless
psi = 0.929 # unitless
Tc = 1e4 # K

# here no need for nH nor ne as they cancel out
def equilibrium1(T,Z,Tc,psi):
    return psi*Tc*k - (0.684 - 0.0416 * np.log(T/(1e4 * Z*Z)))*T*k

# test and visualize first
temps = np.linspace(1, 1e7, 1000)
eqs = equilibrium1(temps, Z, Tc, psi)
# plt.plot(temps, eqs)
# plt.show()

brac1 = [1, 1e7] # K
midpoint1 = 0.5e7 # mid point of the bracket interval
print('----------2a----------')
def time_bisection():
    root1, num_it1 = bisection(equilibrium1, brac1, Z, Tc, psi, abs_tol = 0.01, frac_tol
    = 1e-8, max_iter = 50)
    return root1, num_it1

def time_secant():
    root2, num_it2 = secant(equilibrium1, brac1, Z, Tc, psi, abs_tol = 0.01, frac_tol =
    1e-8, max_iter = 50)
    return root2, num_it2

def time_false_position():
    root3, num_it3 = false_position(equilibrium1, brac1, Z, Tc, psi, abs_tol = 0.01,
    frac_tol = 1e-8, max_iter = 50)
    return root3, num_it3

root1, num_it1 = bisection(equilibrium1, brac1, Z, Tc, psi, abs_tol = 0.01, frac_tol = 1
    e-8, max_iter = 50)
bisection_time = timeit.timeit(lambda : time_bisection(), number = 1)
print('\nroot and number of iterations')
print('using bisection:', root1, num_it1)
print('function evaluated at root:', equilibrium1(root1, Z, Tc, psi))
print('time taken:', bisection_time)

root2, num_it2 = secant(equilibrium1, brac1, Z, Tc, psi, abs_tol = 0.01, frac_tol = 1e
    -8, max_iter = 50)
secant_time = timeit.timeit(lambda : time_secant(), number = 1)
print('\nusing secant method:', root2, num_it2)
print('function evaluated at root:', equilibrium1(root2, Z, Tc, psi))
print('time taken:', secant_time)

root3, num_it3 = false_position(equilibrium1, brac1, Z, Tc, psi, abs_tol = 0.01,
    frac_tol = 1e-8, max_iter = 50)
false_position_time = timeit.timeit(lambda : time_false_position(), number = 1)
print('\nusing false position method:,', root3, num_it3)
```

```
print('function evaluated at root:', equilibrium1(root3, Z, Tc, psi))
print('time_taken', false_position_time)
```

HII–regions.py

We use a bracket of $[1, 10^7]$ K for all three of them. The output is given below:

```
----------2a----------

root and number of iterations
using bisection: 32539.12576294411 30
function evaluated at root: 3.2802042676833513e-20
time taken: 0.00011720135807991028
maximum secant interations reached
maximum secant interations reached

using secant method: nan 50
function evaluated at root: nan
time taken: 0.0003875158727169037

using false position method:, 32539.126737497554 23
function evaluated at root: 0.0
time_taken 0.00026347488164901733
```

HII–regions_output.txt

As we can see, both bisection and false position find the root, while secant diverges. We also see the number of iterations, next to the value of the root. We see that the time taken by bisection is the smallest, and we should use bisection in this case.

2.2 2b: A more complex HII-region

We now have to find the root of the equilibrium2 function, which is a little more complicated. We evaluate all three methods for each value of n_e, and at the end we print how fast the best method converges for the different values.

```
----------2b----------

for ne =  0.0001
root and number of iterations
using bisection: 160647887532832.94 36
function evaluated at root: 1.3118954069051732e-26
maximum secant interations reached
using secant method: nan 100
function evaluated at root: nan
using false position method:, 160647887536816.06 124
function evaluated at root: 0.0

for ne =  1
root and number of iterations
using bisection: 33861.300234646915 69
function evaluated at root: 1.5869127818352802e-35
using secant method: 33861.300235178554 11
function evaluated at root: -9.183549615799121e-41
using false position method:, 33861.298551236556 100000
function evaluated at root: 5.026513802461895e-32

for ne =  10000.0
root and number of iterations
using bisection: 10525.886018968453 70
function evaluated at root: 1.9976650564114478e-31
using secant method: 10525.88601966122 10
function evaluated at root: 2.3553508877120796e-37
using false position method:, 10525.885169143065 100000
function evaluated at root: 2.4525554870977876e-28
------------------------------
Bisection for smallest n took: 0.00018875300884246826
Secant for middle ne took: 0.00010949745774269104
Secant for biggest ne took: 9.28826630115509e-05
```

HII–regions_output.txt

For small n_e, secant diverges. Bisection and false position both find the root, but bisection is fastest. We choose bisection.

For middle n_e, secant does not diverge. It actually finds the root in only 11 iterations. Bisection also works, but with more iterations, and false position takes a very long time to converge. It takes about 100,000 iterations to reach a good minimum, which makes it the slowest.

For largest n_e, secant again finds the root the fastest. False position is still the slowest, and bisection is pretty good.

We conclude that we should choose bisection for the first value n_e, and secant for the other two values.